

LESSON

Cron을 활용한 스케줄링

```
animLength = toTime - fromTime

# Ask user for directory
filePath = c4d.storage.SaveDialog()
filePath, objName = os.path.split(filePath)
objName = objName + ". "
filePath = filePath + "\\ "

# Ask for confirmation
questionDialogText = "Obj Sequence will be saved as:\n\n" \
    "" + filePath + objName + "####.obj\n\n" \
    "From frame " + str(fromTime) + " to " + str(toTime) + " "
proceedBool = c4d.gui.QuestionDialog(questionDialogText)

if proceedBool == True:

    # Loop through animation and export frames
    for x in range(0, animLength):

        # change frame, redraw view
        moveTime = c4d.BaseTime(fromTime, docFps) + c4d.BaseTime(x, docFps)
        doc.SetTime(moveTime)
        c4d.EventAdd(c4d.EVENT_FORCEREDRAW)
        c4d.DrawViews(c4d.DRAWFLAGS_FORCEFULLREDRAW)

        # update status bar
        c4d.StatusSetText("Exporting " + str(x) + " of " + str(animLength))
        c4d.StatusSetBar(100.0*x/animLength)

        # add buffer 0.001
        bufferedNumber = str(doc.GetTime().GetFrame(docFps))
        if len(bufferedNumber) < 3:
```

Cron : 개요

- 유닉스 계열 (Linux, Mac OS 등) 운영체제 컴퓨터에서 시간을 기반으로 한 **작업 예약 스케줄러**
- 주기적으로 고정된 시간, 날짜, 간격에 원하는 프로그램을 실행 할 수 있도록 도와줌
- 서버는 꺼지지 않고 항상 작동 중인 것을 활용하여, 주로 서버에 적용
 - 평소에는 실행되지 않고 대기하다가, 특정 시간이 되면 해당 프로그램을 실행

Cron : 형식

✦ 프로그램 실행을 시간 형식에 맞게 적어 예약

시간 형식

Minute (0 ~ 59) : 분

Hour (0 ~ 23) : 시간

Day (1 ~ 31) : 일

Month (1 ~ 12) : 월

Week (0 ~ 6) : 요일 , 0은 일요일

예시

0 12 1 1 3

1월 1일 수요일 12시
정각에 실행

Cron : 활용

- ☛ 일정 간격의 모든 시간을 지정할 경우 : *

30 * * * * : 매시간 30분마다 실행

0 0 * * * : 매일 0시 0분마다 실행

- ☛ 일정 시간 간격을 반복하도록 지정할 경우 : ,

10,20,30 * * * * : 매시간 10분, 20분, 30분에 실행

Cron : 활용

🔧 시간 범위를 지정할 경우 : -

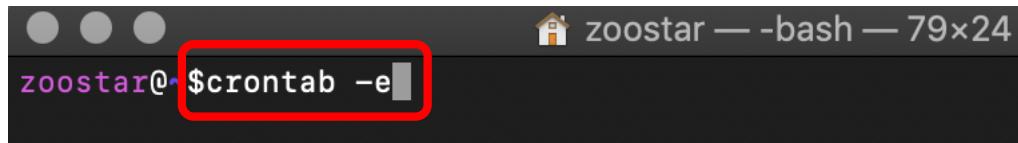
30 13-15 * * * : 13시,14시,15시 30분에 실행

🔧 시간 간격을 지정할 경우: /

0-30/5 * * * * : 매시간 0분부터 30분까지 5분 간격으로 실행

Cron 사용법 : UNIX 계열 운영체제

- 유닉스 계열(Linux, Mac OS 등) 운영체제에서는 별도의 설치없이 기본으로 사용 가능
- 터미널 창(콘솔)에서 **crontab -e** 명령어로 cron 설정 파일 편집
 - 현재 로그인 한 사용자에서 실행



```
zoostar ~ — -bash — 79x24
zoostar@ ~ % crontab -e
```



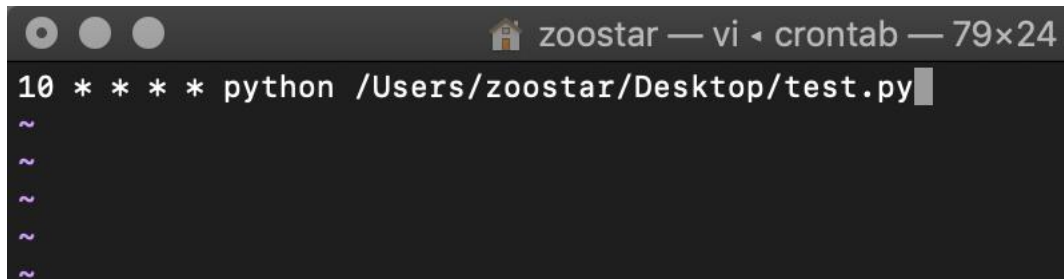
crontab -l : 예약된 작업의 목록 보기

crontab -r : 예약된 작업 삭제

Cron 사용법 : UNIX 계열 운영체제

🔗 작업 스케줄 설정 + 실행할 명령어

- 매 10분마다 바탕화면의 text.py 파이썬 파일 실행

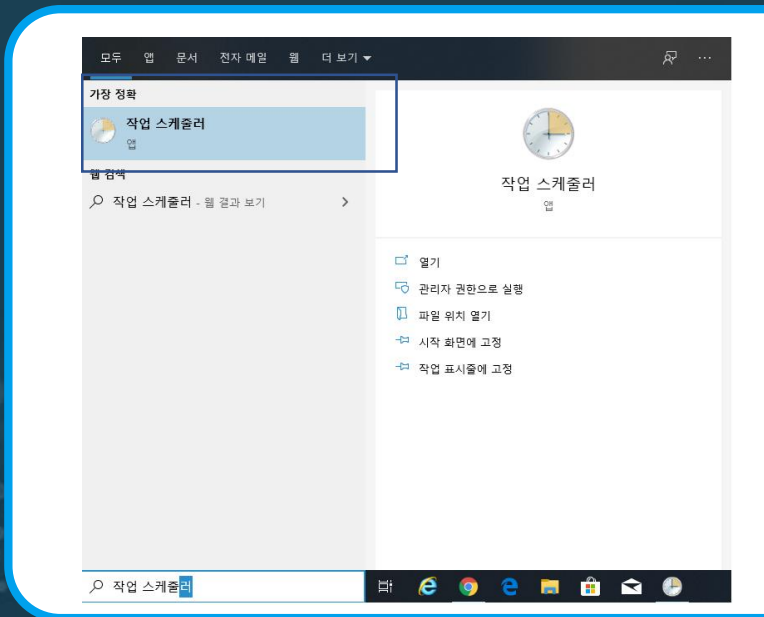
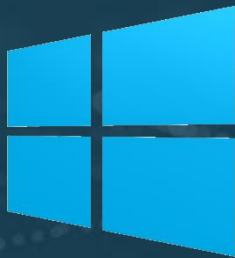


The screenshot shows a terminal window titled "zoostar — vi — crontab — 79x24". The terminal content displays a cron job entry: "10 * * * * python /Users/zoostar/Desktop/test.py". Below this entry, there are four tilde (~) characters, likely representing the rest of the crontab file.

```
zoostar — vi — crontab — 79x24
10 * * * * python /Users/zoostar/Desktop/test.py
~
~
~
~
```

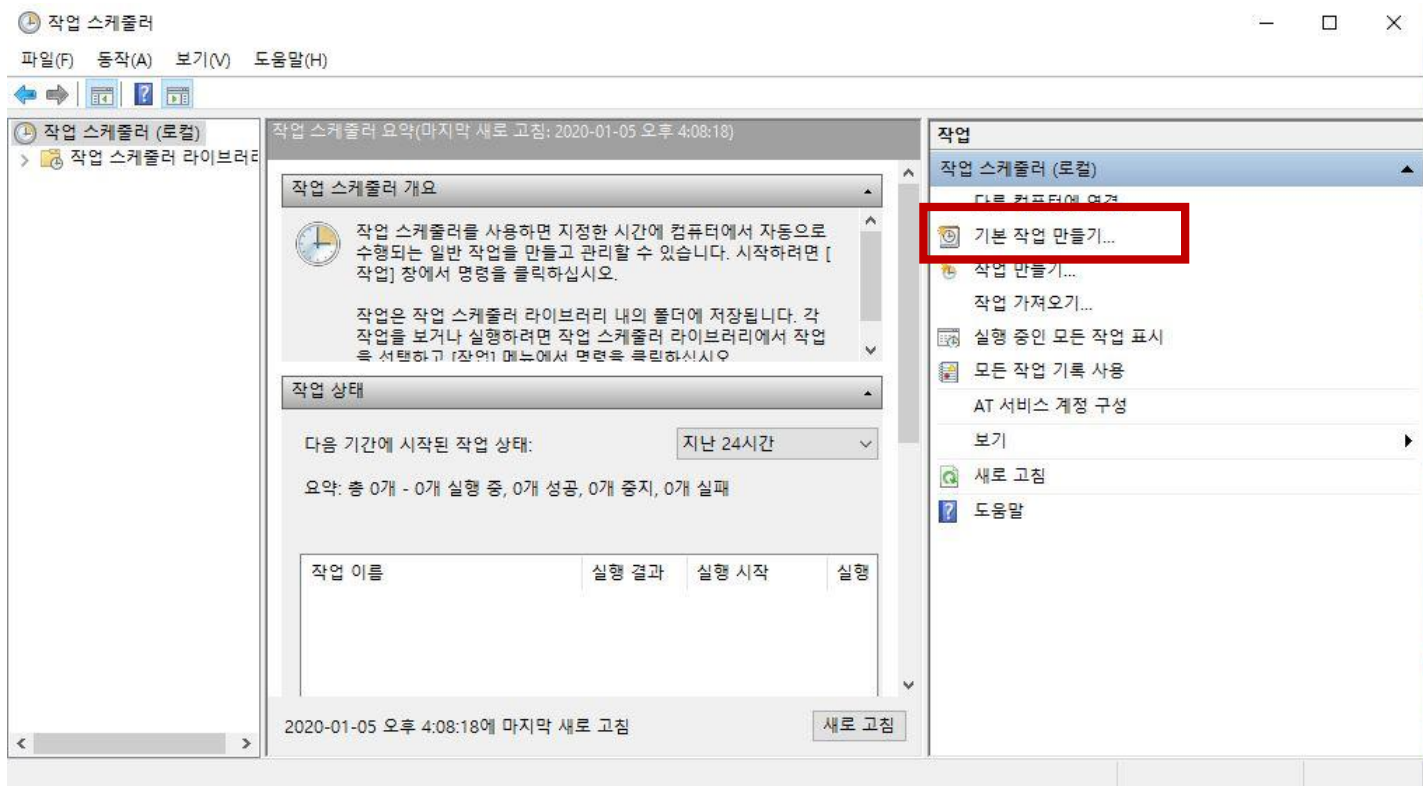
Cron 사용법 : Windows 계열 운영체제

- ❖ 별도의 Cron 프로그램 설치 필요
 - nncron, wincron ...
- ❖ Windows에서 Cron과 같은 역할을 하는 프로그램을 활용
 - 작업 스케줄러
- ❖ 명령어가 아닌 직관적인 GUI 환경으로 사용성이 좋음



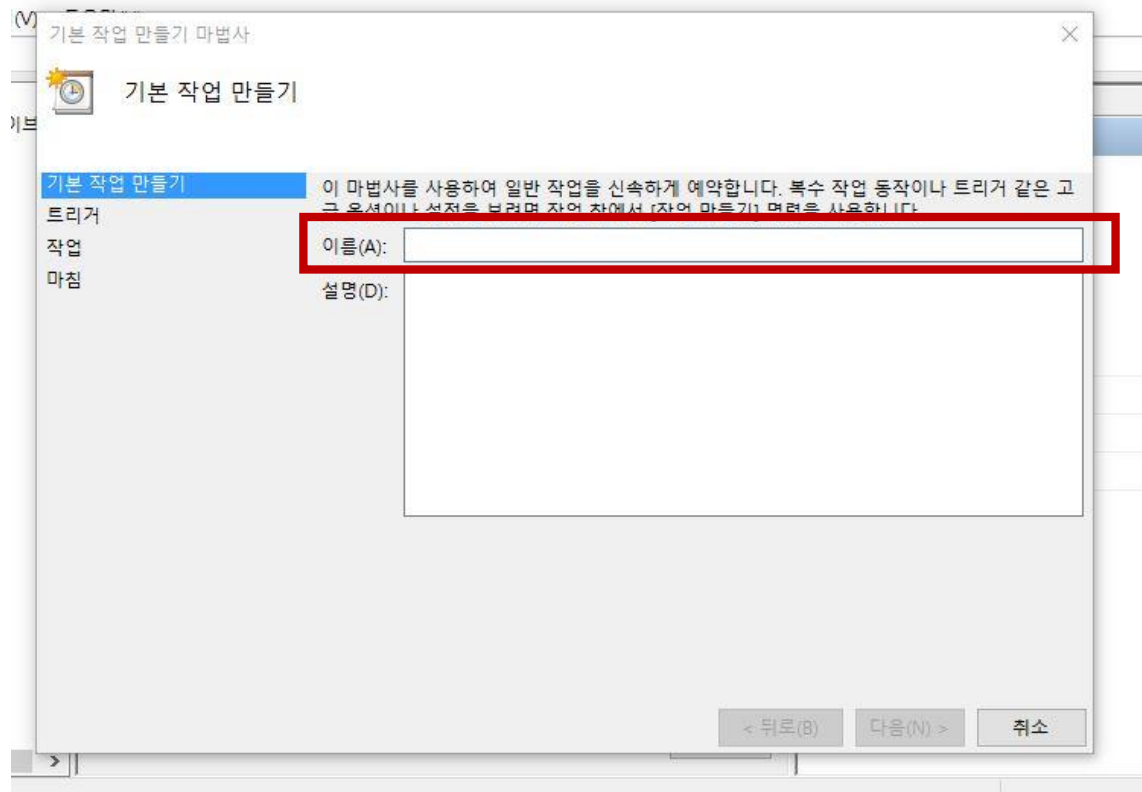
스케줄링 구현하기(Windows) Step1

작업 스케줄러 실행



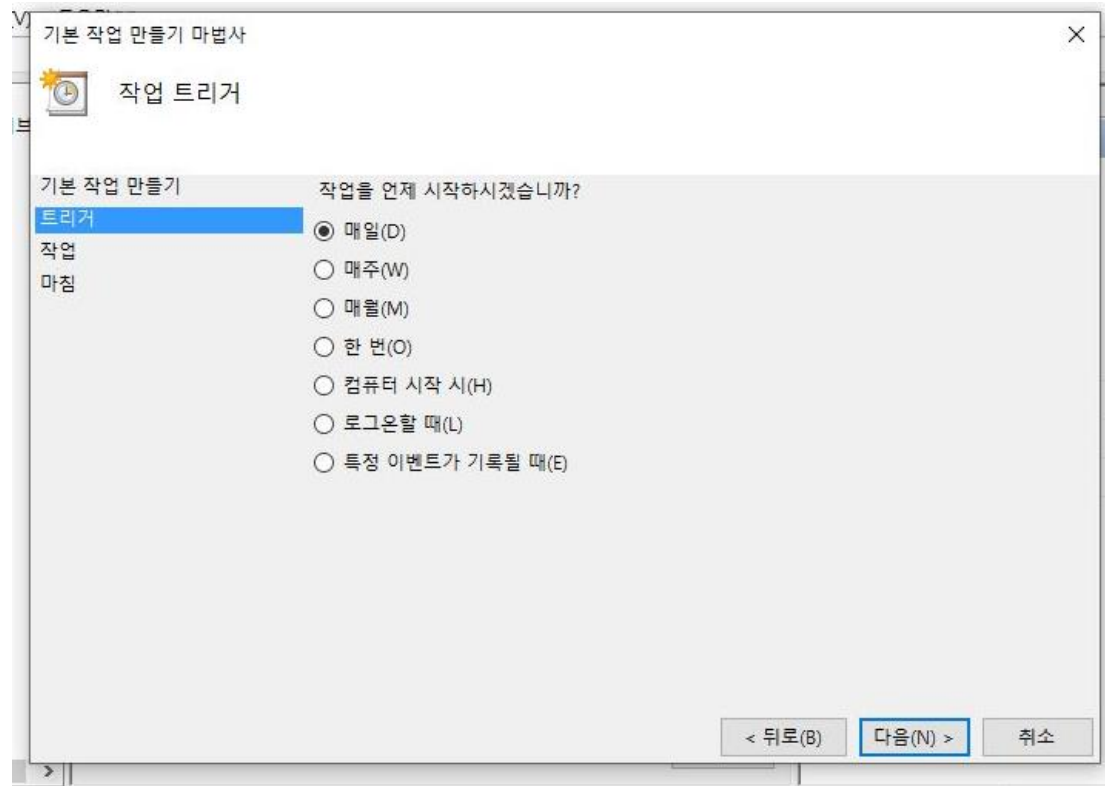
스케줄링 구현하기(Windows) Step1

기본 작업 만들기



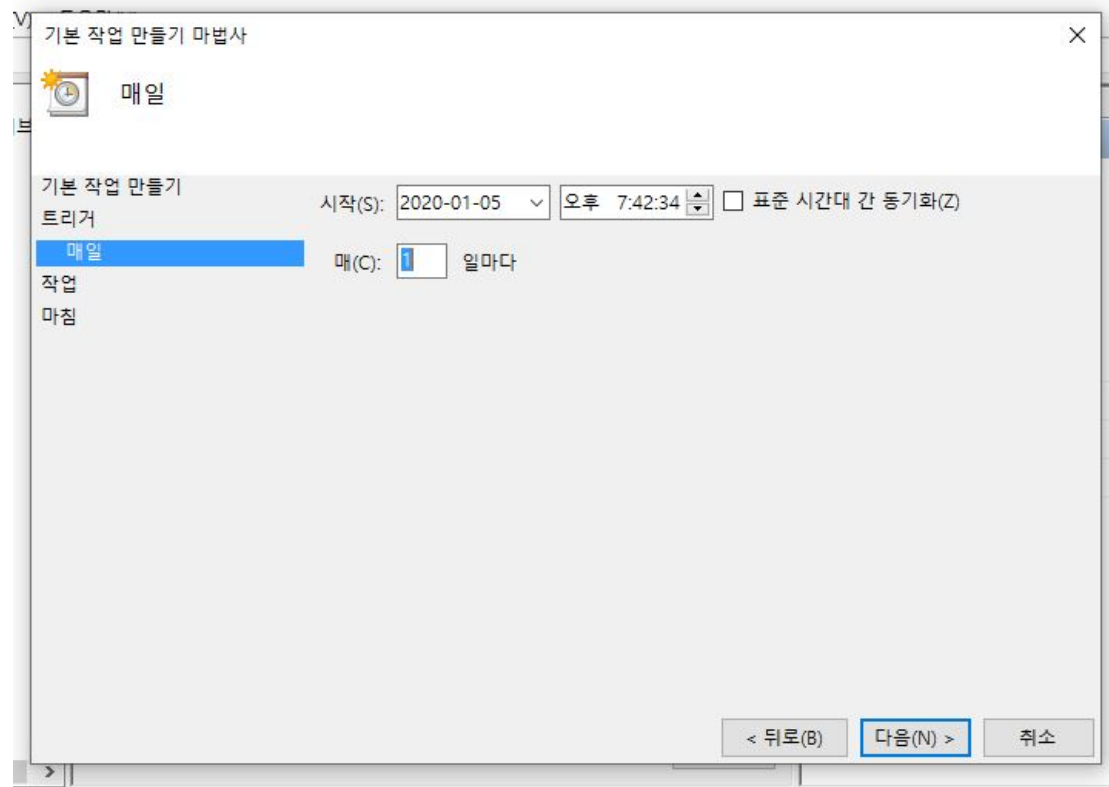
스케줄링 구현하기(Windows) Step2

트리거(실행 조건) 설정



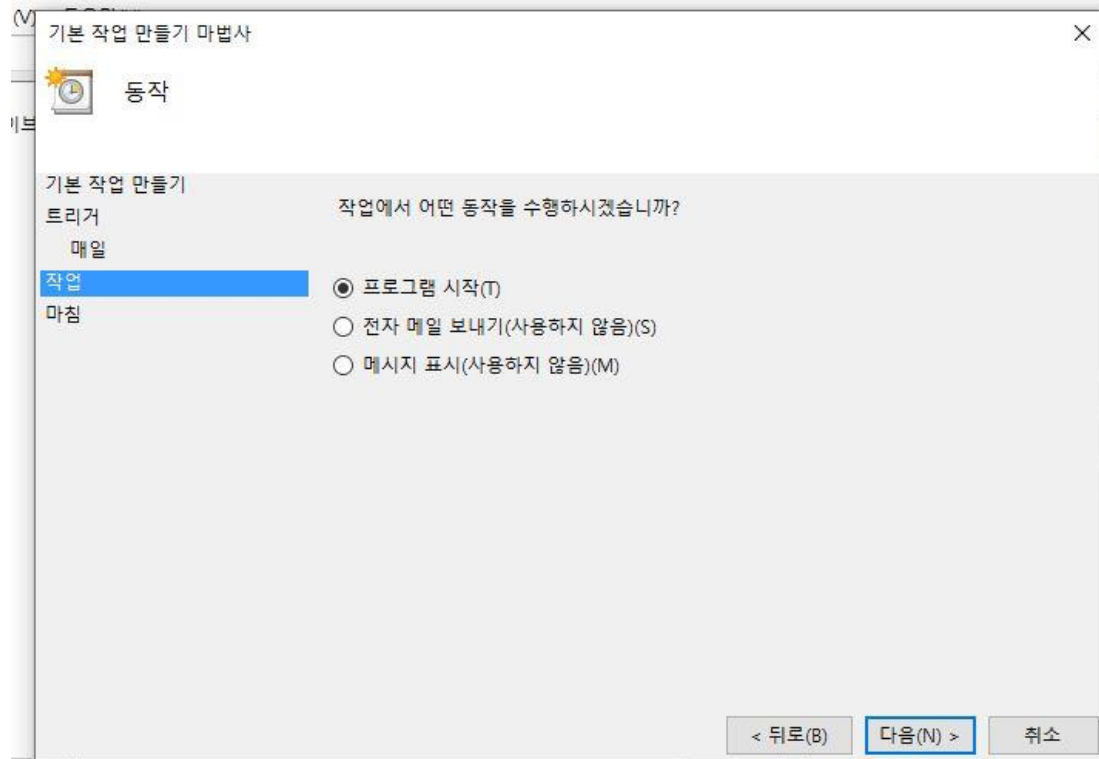
스케줄링 구현하기(Windows) Step2

트리거(실행 조건) 설정



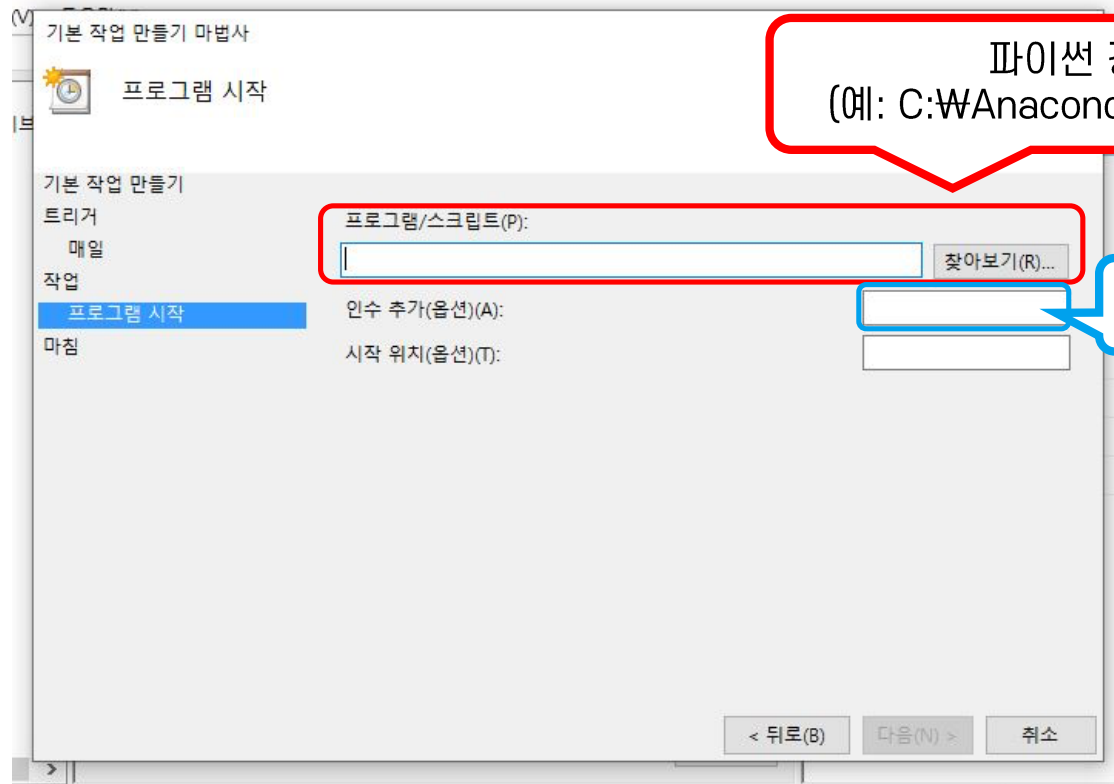
스케줄링 구현하기(Windows) Step3

실행할 작업 선택



스케줄링 구현하기(Windows) Step3

실행 파일 설정



파이썬 경로
(예: C:\Anaconda3\python)

실행 파일 경로

LESSON

파이썬 모듈을 활용한 스케줄링

```
animLength = toTime - fromTime

# Ask user for directory
filePath = c4d.storage.SaveDialog()
filePath, objName = os.path.split(filePath)
objName = objName + ". "
filePath = filePath + "\\ "

# Ask for confirmation
questionDialogText = "Obj Sequence will be saved as:\n\n" \
    "" + filePath + objName + "####.obj\n\n" \
    "From frame " + str(fromTime) + " to " + str(toTime) + " "
proceedBool = c4d.gui.QuestionDialog(questionDialogText)

if proceedBool == True:

    # Loop through animation and export frames
    for x in range(0, animLength):

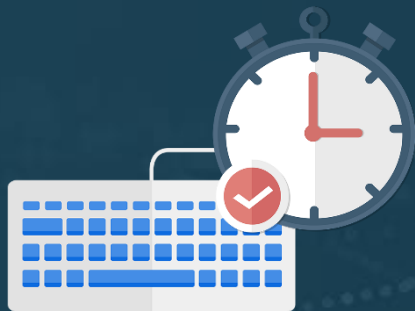
        # change frame, redraw view
        moveTime = c4d.BaseTime(fromTime, docFps) + c4d.BaseTime(x)
        doc.SetTime(moveTime)
        c4d.EventAdd(c4d.EVENT_FORCEREDRAW)
        c4d.DrawViews(c4d.DRAWFLAGS_FORCEFULLREDRAW)

        # update status bar
        c4d.StatusSetText("Exporting " + str(x) + " of " + str(animLength))
        c4d.StatusSetBar(100.0*x/animLength)

        # add buffer 0001
        bufferedNumber = str(doc.GetTime().GetFrame(docFps))
        if len(bufferedNumber) < 4:
```

Schedule 모듈 : 개요

- ❖ 파이썬에서 작업 예약을 하기 위한 모듈로
실행할 코드를 함수로 만들어서 호출하는 방식
- ❖ `pip install schedule` 명령어로 설치
- ❖ 명령어가 직관적으로 간단한 스케줄을 적용하기에 적합
 - 복잡한 스케줄은 작성하기 힘들



Schedule 모듈 : 활용

✦ 다음 세 가지를 조합하여 활용

- `every()`
: 매번 반복

- `seconds` : 초
- `minutes` : 분
- `hour` : 시
- `day` : 일
- `Monday - Sunday` : 요일

- `at()`
: 특정 시간

- 예시 : `스케줄.every().day.at(13:00)` : 매일 13시에 실행

✦ 스케줄 작성 후 `run_pending()` 함수로 시작

Schedule 모듈 : 활용

♣ schedule 구현 예시

무한 루프로
계속 실행 중

In [*]:

```
import schedule
import time

def job():
    print(time.time())

schedule.every(10).seconds.do(job)

while True:
    schedule.run_pending()
```

매 10초마다
함수 실행

```
1578217284.858464
1578217294.85855
1578217304.858618
```

Schedule 모듈 : 활용

- ♣ while 반복문을 활용하여 꾸준히 스케줄을 유지
- ♣ 특정 조건에 만족하는 스케줄이 있을 때마다 해당 함수 실행
- ♣ 해당 파이썬 파일을 계속 실행 시켜 두어야 한다는 단점이 있음

while 반복문을 이용한 스케줄링 예시

```
import schedule

def job():
    print("I'm working...")

#20분 마다 실행
schedule.every(20).minutes.do(job)

#매 시간 실행
schedule.every().hour.do(job)

#매일 10시 30분에 실행
schedule.every().day.at("10:30").do(job)

#매주 월요일 실행
schedule.every().monday.do(job)

#매주 수요일 13시 15분에 실행
schedule.every().wednesday.at("13:15").do(job)

while True:
    schedule.run_pending()
```

APScheduler 모듈 : 개요

- ♣ Advanced Python Scheduler 의 약자로 파이썬 코드를 주기적으로 수행 할 수 있도록 도와주는 외부 모듈
- ♣ MongoDB, Redis 등의 백엔드와 함께 사용할 수 있음(기본은 Memory)
 - 작업을 데이터베이스에 저장한 뒤 불러와서 사용 가능
- ♣ asyncio, gevent 등 여러가지 프레임워크와 연동을 지원
- ♣ pip install apscheduler 명령어로 설치 가능

APScheduler 모듈 : 개요

- ♣ scheduler 모듈과는 다르게 작업의 기간, 일시 정지, 다시 시작, 삭제 등 여러 가지 조건에 따라 스케줄링 가능
- ♣ 총 3가지 수행 방식을 지원

Cron 방식

Cron 표현식으로 코드를 수행

Date 방식

특정 날짜에 코드를 수행

Interval 방식

일정 주기로 코드를 수행

APScheduler 모듈 : 개요

스케줄러의 종류

- ① BlockingScheduler : 단일 작업 수행 시 사용
- ② BackgroundScheduler : 다수 작업 수행 시 사용
- ③ AsyncIOScheduler, GeventScheduler ... : 각 프레임워크 내 작업 수행 시 사용

스케줄러 객체를 생성 →
add_job() 함수를 추가해 작업 예약
→ start() 함수로 스케줄러 시작

```
from apscheduler.schedulers.background import BackgroundScheduler

sched = BackgroundScheduler()

def job():
    print("I'm working...")

sched.add_job(job, 'interval', seconds=3, id="1")

sched.start()

I'm working...
I'm working...
I'm working...
```

APScheduler 모듈 : 수행 방식별 사용법

🔌 **interval 방식** : start_date, end_date(해당 작업의 실행 시작~끝 지정)

```
from apscheduler.schedulers.blocking import BlockingScheduler

def job_function():
    print("Hello World")

sched = BlockingScheduler()

#매 2시간 간격으로 실행
sched.add_job(job_function, 'interval', hours=2)

sched.start()
```

Parameters

- **weeks** (*int*) – number of weeks to wait
- **days** (*int*) – number of days to wait
- **hours** (*int*) – number of hours to wait
- **minutes** (*int*) – number of minutes to wait
- **seconds** (*int*) – number of seconds to wait
- **start_date** (*datetime|str*) – starting point for the interval calculation
- **end_date** (*datetime|str*) – latest possible date/time to trigger on
- **timezone** (*datetime.tzinfo|str*) – time zone to use for the date/time calculations
- **jitter** (*int|None*) – advance or delay the job execution by **jitter** seconds at most.

APScheduler 모듈 : 수행 방식별 사용법

🔗 Cron 방식 : 문자열로도 표현 가능

```
from apscheduler.schedulers.blocking import BlockingScheduler

def job_function():
    print "Hello World"

sched = BlockingScheduler()

#6월부터 8월까지, 11월부터 12월까지 매 셋째 주 금요일, 0시부터 3시까지 한시간 간격으로 실행
sched.add_job(job_function, 'cron', month='6-8,11-12', day='3rd fri', hour='0-3')

sched.start()
```

Parameters

- year (int|str) – 4-digit year
- month (int|str) – month (1-12)
- day (int|str) – day of the (1-31)
- week (int|str) – ISO week (1-53)
- day_of_week (int|str) – number or name of weekday (0-6 or mon,tue,wed,thu,fri,sat,sun)
- hour (int|str) – hour (0-23)
- minute (int|str) – minute (0-59)
- second (int|str) – second (0-59)
- start_date (datetime|str) – earliest possible date/time to trigger on (inclusive)
- end_date (datetime|str) – latest possible date/time to trigger on (inclusive)
- timezone (datetime.tzinfo|str) – time zone to use for the date/time calculations (defaults to scheduler timezone)
- jitter (int|None) – advance or delay the job execution by `jitter` seconds at most.

APScheduler 모듈 : 수행 방식별 사용법

🔗 **date 방식** : 특정 시간에 작업 수행, args로 매개변수 전달

```
from apscheduler.schedulers.blocking import BlockingScheduler

sched = BlockingScheduler()

def my_job(text):
    print(text)

# 2020년 12월 31일 정각에 text 문구 출력
sched.add_job(my_job, 'date', run_date=date(2020, 12, 31), args=['text'])

sched.start()
```

Parameters

- **run_date** (*datetime|str*) – the date/time to run the job at
- **timezone** (*datetime.tzinfo|str*) – time zone for **run_date** if it doesn't have one already

LESSON

조회수 데이터 가져오기 (Selenium 모듈)

```
animLength = toTime - fromTime

# Ask user for directory
filePath = c4d.storage.SaveDialog()
filePath, objName = os.path.split(filePath)
objName = objName + "_"
filePath = filePath + "\\ "

# Ask for confirmation
questionDialogText = "Obj Sequence will be saved as:\n\n" \
    "" + filePath + objName + "###.obj\n\n" \
    "From frame " + str(fromTime) + " to " + str(toTime) + " "
proceedBool = c4d.gui.QuestionDialog(questionDialogText)

if proceedBool == True:

    # Loop through animation and export frames
    for x in range(0,animLength):

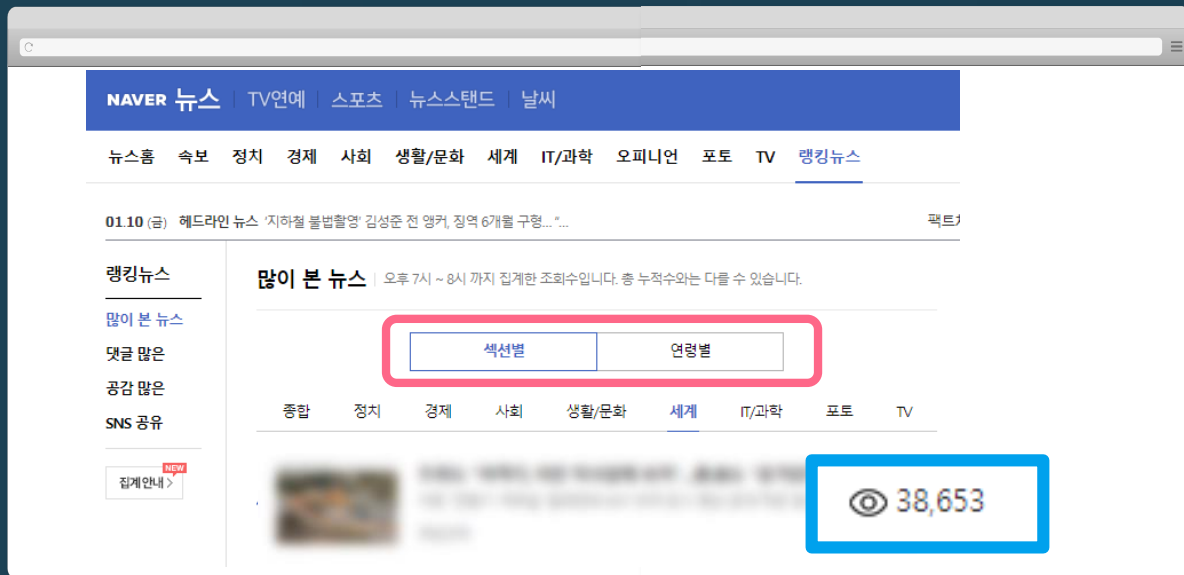
        # change frame, redraw view
        moveTime = c4d.BaseTime(fromTime,docFps) + c4d.BaseTime(x,docFps)
        doc.SetTime(moveTime)
        c4d.EventAdd(c4d.EVENT_FORCEREDRAW)
        c4d.DrawViews(c4d.DRAWFLAGS_FORCEFULLREDRAW)

        # update status bar
        c4d.StatusSetText("Exporting " + str(x) + " of " + str(animLength))
        c4d.StatusSetBar(100.0*x/animLength)

        # get buffer pool
        bufferedNumber = str(doc.GetTime().GetFrame(docFps))
        if len(bufferedNumber) > 10:
```

‘랭킹 뉴스’ 페이지 접근해 데이터 가져오기

- Naver 뉴스의 많이 본 뉴스(랭킹 뉴스) 페이지에서
섹션별, 연령별 조회수 비교를 위한 데이터 가져오기



1. 랭킹 뉴스 페이지 접근 Step1

Naver 뉴스 페이지에 접근한다.

The screenshot shows the Naver homepage with the DevTools console open. The console displays the HTML structure of the page, specifically the navigation bar. A red arrow points from the 'a.an_a.mn_news' link in the console to the '뉴스' link in the navigation bar.

NAVER whale 인터넷의 새로운 시작! 네이버 웨일로 차원이 다른 웹서핑을 경험해보세요!

NAVER

a.an_a.mn_news 28x16

메일 카페 블로그 지식IN 쇼핑 Pay TV 사전 뉴스 증권 부동산 지도 영화 뮤직 책 웹툰 더보기

신년맞이 1시간 초특가!

```
<div class="u_skip"></div>
<!-- //소립 내비게이션 -->
<!-- 크롬/웨일/엣지 -->
<div id="whale_promotion_banner" class="banner_area type_chrome" style="display: block; overflow: visible; height: 48px; position: relative;"></div>
<a id="whale_promotion_download_file" style="display: none" href="http://update.whale.naver.net/downloads/installers/WhaleSetup.exe" download></a>
<div id="PM_ID_ct" class="wrap">
  <!-- 헤더 -->
  <div class="header" role="banner">
    <div class="special_bg"></div>
    <div class="section_navbar">
      <div class="area_navigation" role="navigation">
        <ul class="an_l"></ul>
        <ul id="PM_ID_serviceNavi" class="an_l">
          <li class="an_item"></li>
          <li class="an_item">
            <a href="https://news.naver.com/" class="an_a mn_news" data-clk="svc.news">
              <span class="an_icon"></span>
              <span class="an_txt">뉴스</span>
            </a>
          </li>
        </ul>
      </div>
    </div>
  </div>
```

1. 랭킹 뉴스 페이지 접근 Step1

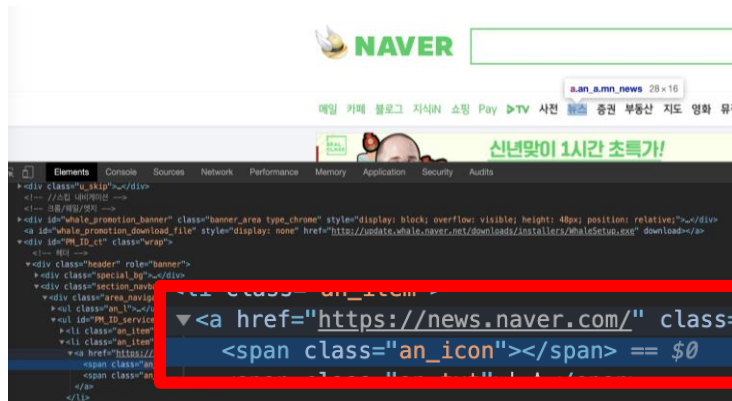
Naver 뉴스 페이지에 접근한다.

크롬 웹 드라이버 설치 경로

```
import selenium
import time
from selenium import webdriver
```

```
path = "/Users/zoostar/Downloads/chromedriver"
driver = webdriver.Chrome(path)
driver.get('https://www.naver.com')
```

```
element = driver.find_element_by_class_name("mn_news")
element.click()
```



1. 랭킹 뉴스 페이지 접근 Step2

랭킹 뉴스 페이지에 접근한다.



```
<div id="lnb" class="lnb_menu">
<li class="lnb_menu_home on"><a href="/main/home.nhn" class="nclicks(LNB.home)"><span class="tx">뉴스</span> <span class="blind">선택됨</span></a></li>
<li><a href="/main/list.nhn?mode=LS&mid=sec&sid1=001" class="nclicks(LNB.flash)"><span class="tx">속보</span></a></li>
<li><a href="/main/main.nhn?mode=LS&mid=shms&sid1=100" class="nclicks(LNB.pol)"><span class="tx">정치</span></a></li>
<li><a href="/main/main.nhn?mode=LS&mid=shms&sid1=101" class="nclicks(LNB.eco)"><span class="tx">경제</span></a></li>
<li><a href="/main/main.nhn?mode=LS&mid=shms&sid1=102" class="nclicks(LNB.soc)"><span class="tx">사회</span></a></li>
<li><a href="/main/main.nhn?mode=LS&mid=shms&sid1=103" class="nclicks(LNB.lif)"><span class="tx">생활/문화</span></a></li>
<li><a href="/main/main.nhn?mode=LS&mid=shms&sid1=104" class="nclicks(LNB.wor)"><span class="tx">세계</span></a></li>
<li><a href="/main/main.nhn?mode=LS&mid=shms&sid1=105" class="nclicks(LNB.sci)"><span class="tx">IT/과학</span></a></li>
<li><a href="/main/opinion/home.nhn" class="nclicks(LNB.opi)"><span class="tx">오피니언</span></a></li>
<li><a href="/main/photo/index.nhn?mid=pho" class="nclicks(LNB.pho)"><span class="tx">포토</span></a></li>
<li><a href="/main/tv/index.nhn?mid=tyh" class="nclicks(LNB.tv)"><span class="tx">TV</span></a></li>
<li><a href="/main/ranking/popularDay.nhn?mid=etc&sid1=111" class="nclicks(LNB.ranking)"><span class="tx">랭킹뉴스</span></a></li>
</ul>
<form name="lnb_searchForm" id="lnb_searchForm" method="get" action="https://search.naver.com/search.naver" target="_blank" accept-charset="UTF-8">
<fieldset>
<legend>뉴스 검색</legend>
<input type="text" title="뉴스 검색" name="query" accesskey="s" class="text_index" style="ime-mode:active;" />
<input type="hidden" name="where" value="news">
<input type="hidden" name="ie" value="utf8">
<input type="hidden" name="sm" value="nws_hyt">
<button type="submit" class="btn_search_lnb nclicks(LNB.search)"><span class="tx"><span class="blind">검색</span></span></button>
</fieldset>
</form>
</div>
</div>
```

1. 랭킹 뉴스 페이지 접근 Step2

랭킹 뉴스 페이지에 접근한다.

```

ss="nclicks(LNB.home)"><span class="tx">뉴스홈</span> <span class="tx">속보</span></a></li>
ass="nclicks(LNB.flash)"><span class="tx">속보</span></a></li>
ass="nclicks(LNB.pol)"><span class="tx">정치</span></a></li>
ass="nclicks(LNB.eco)"><span class="tx">경제</span></a></li>
ass="nclicks(LNB.soc)"><span class="tx">사회</span></a></li>
ass="nclicks(LNB.lif)"><span class="tx">생활/문화</span></a></li>
ass="nclicks(LNB.wor)"><span class="tx">세계</span></a></li>
ass="nclicks(LNB.sci)"><span class="tx">IT/과학</span></a></li>
pi)"><span class="tx">오피니언</span></a></li>
s(LNB.pho)"><span class="tx">포토</span></a></li>
NB.tv)"><span class="tx">TV</span></a></li>
l" class="nclicks(LNB.ranking)"><span class="tx">랭킹뉴스</span></a></li>

action="https://search.naver.com/search.naver" target="_blank"

key="s" class="text_index" style="ime-mode:active;"/>

NB.search)"><span class="tx"><span class="blind">검색</span></span></a></li>
    
```



랭킹 뉴스 카테고리의 class는 'tx'로 되어 있으며 동일한 클래스의 요소 중 뒤에서 두 번째에 위치해 있는 것을 확인

```

import selenium
import time
from selenium import webdriver
    
```

```

path = "/Users/zoostar/Downloads/chromedriver"
driver = webdriver.Chrome(path)
driver.get('https://www.naver.com')
    
```

```

element = driver.find_element_by_class_name("mn_news")
    
```

```

element = driver.find_element_by_id("lbn").find_elements_by_class_name("tx")
element[len(element)-2].click()
    
```

2. 섹션별 페이지 접근 Step1

섹션별 데이터 규칙을 찾는다.

01.05 (일) 헤드라인 뉴스 한겨레 "유리해, 정치인들 만들 것" ... 불친사 놓고 '영장'

랭킹뉴스

많이 본 뉴스

댓글 많은

공감 많은

SNS 공유

많이 본 뉴스 | 오후 9시 ~ 10시 까지 집계한 조화수입니다. 총 누적수와는 다를 수 있습니다.

li.ranking_category_item.is_selected 58.5 x 37 명별

종합 정치 경제 사회 생활/문화 세계 IT/과학

Elements Console Sources Network Performance Memory Application Security Audits

```

<td class="snb"></td>
<td class="content">
  <div class="content">
    <div class="list_header ranking_header"></div>
    <div class="ranking_type"></div>
    <div class="ranking_category">
      ::before
      <div class="ranking_category_inner">
        <ul class="ranking_category_list">
          <li class="ranking_category_item"></li>
          <li class="ranking_category_item is_selected">
            ::before
            <a href="/main/ranking/popularDay.nhn?rankingType=popular_day&sectionId=100&date=20200105" class="nclinks(rank.tabsec2)"></a> == $0
          </li>
          <li class="ranking_category_item"></li>
          <li class="ranking_category_item"></li>
          <li class="ranking_category_item"></li>
          <li class="ranking_category_item"></li>
          <li class="ranking_category_item"></li>
          <li class="ranking_category_item"></li>
          <li class="ranking_category_item"></li>
        </ul>
      </div>
    </div>
  </td>
</tr>

```

2. 섹션별 페이지 접근 Step1 섹션별 데이터 규칙을 찾는다.

```
import selenium
import time
from selenium import webdriver
```

```
path = "/Users/zoostar/Downloads/chromedriver"
driver = webdriver.Chrome(path)
driver.get('https://www.naver.com')
```

```
element = driver.find_element_by_class_name("mn_news")
element.click()
element = driver.find_element_by_id("lnb").find_elements_by_class_name("tx")
element[len(element)-2].click()
```

#정치, 경제, 사회, 생활/문화, 세계, IT/과학, 포토, TV

```
element = driver.find_elements_by_class_name("ranking_category_item")
```

```
><div class="list_header ranking_header">...</div>
><div class="ranking_type">...</div>
▼<div class="ranking_category">
  ::before
  ▼<div class="ranking_category_inner">
    ▼<ul class="ranking_category_list">
      ▶<li class="ranking_category_item">...</li>
      ▼<li class="ranking_category_item is_selected">
        ::before
```

2. 섹션별 페이지 접근 Step2

반복문을 활용하여 섹션별 페이지 소스를 BeautifulSoup 객체로 저장한다.

```
import selenium
import time
from selenium import webdriver
from bs4 import BeautifulSoup

path = "/Users/zoostar/Downloads/chromedriver"
driver = webdriver.Chrome(path)
driver.get('https://www.naver.com')

element = driver.find_element_by_class_name("mn_news")
element.click()
element = driver.find_element_by_id("lnb").find_elements_by_class_name("tx")
element[len(element)-2].click()

#정치, 경제, 사회, 생활/문화, 세계, IT/과학, 포토, TV
news_list = {}
element = driver.find_elements_by_class_name("ranking_category_item")

for i in range(1, len(element)):
    element = driver.find_elements_by_class_name("ranking_category_item")
    element[i].click()
    soup = BeautifulSoup(driver.page_source, 'html.parser')
    news_list[soup.title.string.split(',')[0]] = soup
    time.sleep(1)
```

종합

정치

경제

사회

생활/문화

세계

IT/과학

포토

TV

```
print(soup.title.string)
```

TV,

많이 본 뉴스 : 네이버 뉴스

3. 연령별 페이지 접근 Step1

연령별 데이터 섹션에 접근한다.



연령별 페이지는 섹션별 탭
다음에 위치한 탭을 클릭해야
나타나기 때문에 해당 요소를
클릭 할 수 있도록 요소 검색

01.05 (일) 헤드라인 뉴스 文대통령도 보낸 축하화환...사람 화환 아무것도 안보낸 ...

랭킹뉴스

많이 본 뉴스

댓글 많은

공감 많은

SNS 공유

많이 본 뉴스 | 오후 9시 ~ 10시 까지 집계한 조화수입니다. 총 누적수와는 다를 수 있습니다.

ul.ranking_type_list 327 x 34

섹션별

연령별

10대

20대

30대

40대

50대

60대 이상

```

Elements Console Sources Network Performance Memory Application Security Audits
<tbody>
  <tr>
    <td class="snb">...</td>
    <td class="content">
      <div class="content">
        <div class="list_header_ranking_header">...</div>
        <div class="ranking_type">
          <ul class="ranking_type_list">
            <li class="ranking_type_item nclicks(rnk.tabsec)">...</li>
            <li class="ranking_type_item is_selected nclicks(rnk.tabage)">
              <a href="/main/ranking/popularDay.nhn?rankingType=age">...</a> == $0
            </li>
          </ul>
        </div>
      </div>
    </td>
  </tr>
</tbody>
    
```

3. 연령별 페이지 접근 Step1

연령별 데이터 섹션에 접근한다.

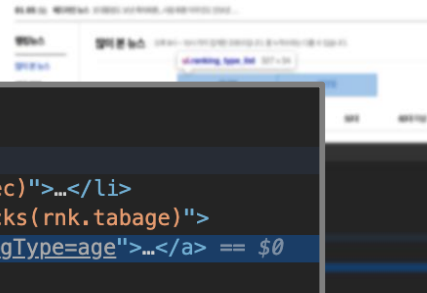
```
import selenium
import time
from selenium import webdriver
from bs4 import BeautifulSoup
```

```
path = "/Users/zoostar/Downloads/chromedriver"
driver = webdriver.Chrome(path)
driver.get('https://www.naver.com')
```

```
element = driver.find_element_by_class_name("mn_news")
element.click()
element = driver.find_element_by_id("lnb").find_elements_by_class_name("tx")
element[len(element)-2].click()
```

```
element = driver.find_elements_by_class_name("ranking_type_item")
element[1].click()
```

```
><div class="list_header ranking_header">...</div>
▼<div class="ranking_type">
  ▼<ul class="ranking_type_list">
    ▶<li class="ranking_type_item nclicks(rnk.tabsec)">...</li>
    ▼<li class="ranking_type_item is_selected nclicks(rnk.tabage)">
      ▶<a href="/main/ranking/popularDay.nhn?rankingType=age">...</a> == $0
    </li>
  </ul>
  </div>
```



3. 연령별 페이지 접근 Step2

반복문을 활용하여 섹션별 페이지 소스를 BeautifulSoup 객체로 저장한다.

```
import selenium
import time
from selenium import webdriver
from bs4 import BeautifulSoup

path = "/Users/zoostar/Downloads/chromedriver"
driver = webdriver.Chrome(path)
driver.get('https://www.naver.com')

element = driver.find_element_by_class_name("mn_news")
element.click()
element = driver.find_element_by_id("lnb").find_elements_by_class_name("tx")
element[len(element)-2].click()

element = driver.find_elements_by_class_name("ranking_type_item")
element[1].click()

age_list = {}
element = driver.find_elements_by_class_name("ranking_category_item")
for i in range(len(element)):
    element = driver.find_elements_by_class_name("ranking_category_item")
    element[i].click()
    soup = BeautifulSoup(driver.page_source, 'html.parser')
    age_list[soup.find('ul', class_='ranking_category_list').find('li', class_='is_selected')
                .find('a').text.split('선택됨')[0].strip()] = soup
    time.sleep(1)
```

3. 연령별 페이지 접근 Step2

반복문을 활용하여 섹션별 페이지 소스를 BeautifulSoup 객체로 저장한다.

```
soup = BeautifulSoup(driver.page_source, 'html.parser')
age_list[soup.find('ul', class_='ranking_category_list').find('li', class_='is_selected')
            .find('a').text.split('선택됨')[0].strip()] = soup
time.sleep(1)
```

```
soup.find('ul', class_='ranking_category_list')
<ul class="ranking_category_list">
<li class="ranking_category_item">
<a class="nclicks(rank.tabage2)" href="/main/ranking/popularDay.nhn?rankingType=age&subType=10">10대 <span class="blind">선택됨</span></a>
</li>
<li class="ranking_category_item">
<a class="nclicks(rank.tabage2)" href="/main/ranking/popularDay.nhn?rankingType=age&subType=20">20대 <span class="blind">선택됨</span></a>
</li>
<li class="ranking_category_item">
<a class="nclicks(rank.tabage2)" href="/main/ranking/popularDay.nhn?rankingType=age&subType=30">30대 <span class="blind">선택됨</span></a>
</li>
<li class="ranking_category_item">
<a class="nclicks(rank.tabage2)" href="/main/ranking/popularDay.nhn?rankingType=age&subType=40">40대 <span class="blind">선택됨</span></a>
</li>
<li class="ranking_category_item">
<a class="nclicks(rank.tabage2)" href="/main/ranking/popularDay.nhn?rankingType=age&subType=50">50대 <span class="blind">선택됨</span></a>
</li>
<li class="ranking_category_item is_selected">
<a class="nclicks(rank.tabage2)" href="/main/ranking/popularDay.nhn?rankingType=age&subType=60">60대 이상 <span class="blind">선택됨</span></a>
</li>
</ul>
```

LESSON

데이터 파싱 및 가공 (BeautifulSoup 모듈)

```
animLength = toTime - fromTime

# Ask user for directory
filePath = c4d.storage.SaveDialog()
filePath, objName = os.path.split(filePath)
objName = objName + ".obj"
filePath = filePath + "\\ "

# Ask for confirmation
questionDialogText = "Obj Sequence will be saved as:\n\n" \
    "" + filePath + objName + "####.obj\n\n" \
    "From frame " + str(fromTime) + " to " + str(toTime) + " "
proceedBool = c4d.gui.QuestionDialog(questionDialogText)

if proceedBool == True:

    # Loop through animation and export frames
    for x in range(0,animLength):

        # change frame, redraw view
        moveTime = c4d.BaseTime(fromTime,docFps) + c4d.BaseTime(x)
        doc.SetTime(moveTime)
        c4d.EventAdd(c4d.EVENT_FORCEREDRAW)
        c4d.DrawViews(c4d.DRAWFLAGS_FORCEFULLREDRAW)

        # update status bar
        c4d.StatusSetText("Exporting " + str(x) + " of " + str(animLength))
        c4d.StatusSetBar(100.0*x/animLength)

        # add buffer 0001
        bufferedNumber = str(doc.GetTime().GetFrame(docFps))
        if len(bufferedNumber) < 4:
```

1. 섹션별 조회수 파싱 Step1 조회수 패턴을 찾는다.

많이 본 뉴스 | 오후 9시 ~ 10시 까지 집계한 조회수입니다. 총 누적수는 다를 수 있습니다.

The screenshot shows a news website with a list of news items. The first item is highlighted with a red box. The view count for this item is 46,246. A blue arrow points from this item to a zoomed-in view of the HTML structure.

The zoomed-in view shows the HTML structure for the first news item. The HTML is as follows:

```


46,246


```

```
news_list['정치'].find('div', class_="ranking_view")
```

```
<div class="ranking_view">46,246</div>
```

1. 섹션별 조회수 파싱 Step2 반복문을 활용한 하나의 섹션 전체 조회수를 파싱한다.

```
print(news_list['정치'].find('div',class_="ranking_view").string)
```

46,246

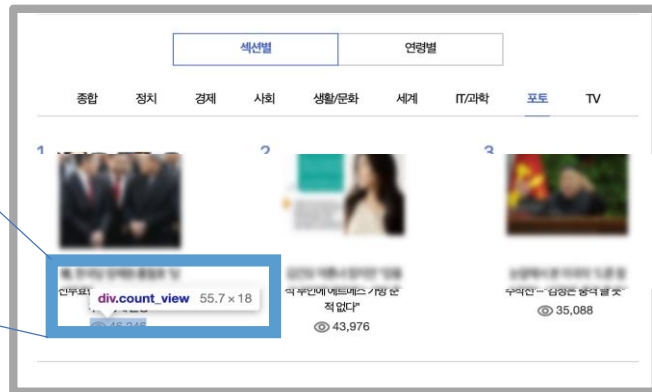
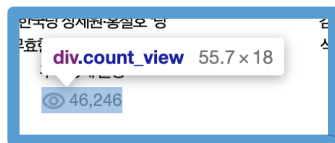
```
sum_view = 0
for i in news_list['정치'].find_all('div',class_="ranking_view"):
    sum_view += int(i.string.replace(",",""))
print(sum_view)
```

329730

1. 섹션별 조회수 파싱 Step3 반복문을 활용한 전체 섹션 전체 조회수를 파싱한다.

```
news_view_list = {}
for i in news_list:
    sum_view = 0
    if(i!='포토' and i!='TV'):
        for j in news_list[i].find_all('div',class_='ranking_view'):
            sum_view += int(j.string.replace(",",""))
    else:
        for j in news_list[i].find_all('div',class_='count_view'):
            sum_view += int(j.string.replace(",",""))
    news_view_list[i] = sum_view
print(news_view_list)
```

```
{'정치': 329730, '경제': 118733, '사회': 370581, '생활/문화': 174381, '세계': 265289, 'IT/과학': 56683, '포토': 517171, 'TV': 368719}
```



2. 연령별 조회수 파싱 Step1

반복문을 활용한 하나의 섹션 전체 조회수를 파싱한다.

많이 본 뉴스 | 오후 9시 ~ 10시까지 집계한 조회수입니다. 총 누적수와는 다를 수 있습니다.

색선별

연령별

10대

20대

30대

40대

50대

60대 이상

1

1,783

가오늘(5일)...

2

1,277

대가오

3

167

는 뜻

4

158

해 (워싱턴=연...

div ranking_view

1,783

```
print(age_list['10대'].find('div',class_="ranking_view"))
```

<div class="ranking_view">1,783</div>

2. 연령별 조회수 파싱 Step2 반복문을 활용한 하나의 연령별 조회수를 파싱한다.

```
age_view_list = {}
for i in age_list:
    sum_view = 0
    for j in age_list[i].find_all('div', class_='ranking_view'):
        sum_view += int(j.string.replace(",", ''))

    age_view_list[i] = sum_view
print(age_view_list)
```

```
{'10대': 3951, '20대': 21984, '30대': 50417, '40대': 34213, '50대': 20125, '60대 이상': 16521}
```

LESSON

분석 결과 시각화 (matplotlib 모듈)

```
animLength = toTime - fromTime

# Ask user for directory
filePath = c4d.storage.SaveDialog()
filePath, objName = os.path.split(filePath)
objName = objName + ". "
filePath = filePath + "\\ "

# Ask for confirmation
questionDialogText = "Obj Sequence will be saved as:\n\n" \
    "" + filePath + objName + "####.obj\n\n" \
    "From frame " + str(fromTime) + " to " + str(toTime) + " "
proceedBool = c4d.gui.QuestionDialog(questionDialogText)

if proceedBool == True:

    # Loop through animation and export frames
    for x in range(0, animLength):

        # change frame, redraw view
        moveTime = c4d.BaseTime(fromTime, docFps) + c4d.BaseTime(x)
        doc.SetTime(moveTime)
        c4d.EventAdd(c4d.EVENT_FORCEREDRAW)
        c4d.DrawViews(c4d.DRAWFLAGS_FORCEFULLREDRAW)

        # update status bar
        c4d.StatusSetText("Exporting " + str(x) + " of " + str(animLength))
        c4d.StatusSetBar(100.0*x/animLength)

        # add buffer 0001
        bufferedNumber = str(doc.GetTime().GetFrame(docFps))
        if len(bufferedNumber) < 4:
```

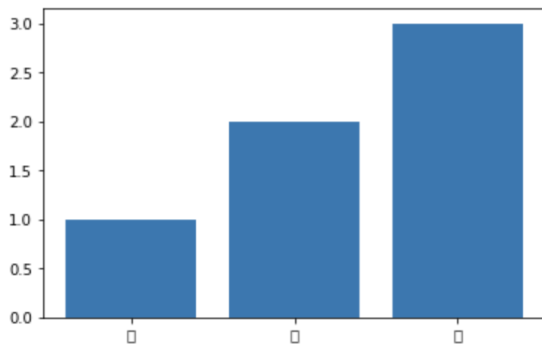
matplotlib 모듈을 이용한 데이터 시각화

- matplotlib 모듈을 이용해 그래프 등을 구현할 경우
한글로 된 글자가 제대로 보이지 않는 문제가 있음
→ 한글이 지원되는 폰트로 변경을 먼저 해주어야 함

```
import matplotlib
import matplotlib.pyplot as plt
%matplotlib inline

figure = plt.figure()
axes = figure.add_subplot(111)

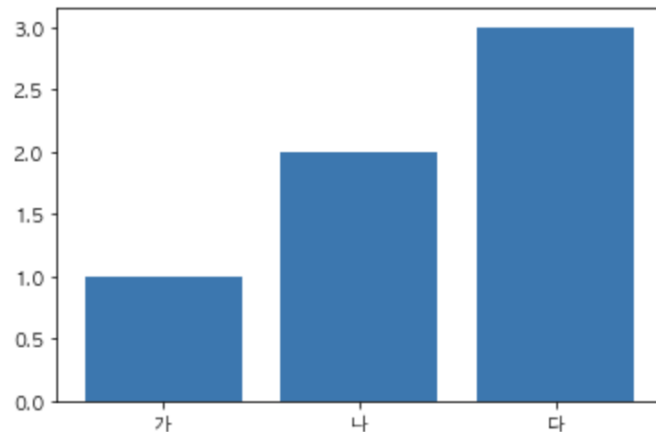
x = ['가', '나', '다']
y = [1, 2, 3]
axes.bar(x, y)
```



1. 한글 폰트 설정하기 matplotlib.rc 함수로 font를 한글이 지원되는 폰트로 변경한다.

```
import matplotlib
import matplotlib.pyplot as plt
%matplotlib inline
matplotlib.rc('font',family='AppleGothic')
figure = plt.figure()
axes = figure.add_subplot(111)

x = ['가', '나', '다']
y = [1, 2, 3]
axes.bar(x, y)
```

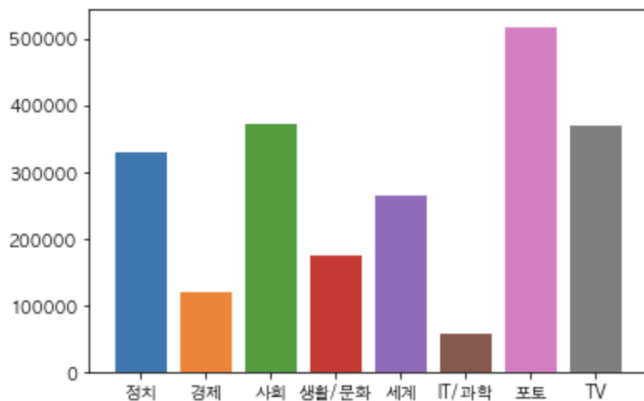


2. 데이터 시각화 섹션별, 연령별 조회수 데이터를 활용해 각각 차트를 만든다.

```
figure = plt.figure()
axes = figure.add_subplot(111)

for i in news_view_list:
    axes.bar(i,news_view_list[i])

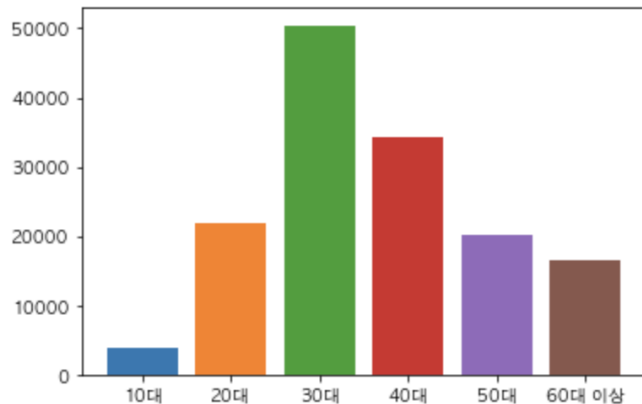
plt.show()
```



```
figure = plt.figure()
axes = figure.add_subplot(111)

for i in age_view_list:
    axes.bar(i,age_view_list[i])

plt.show()
```



LESSON

날씨 데이터 파싱 및 가공

```
animLength = toTime - fromTime

# Ask user for directory
filePath = c4d.storage.SaveDialog()
filePath, objName = os.path.split(filePath)
objName = objName + ". "
filePath = filePath + "\\ "

# Ask for confirmation
questionDialogText = "Obj Sequence will be saved as:\n\n"\
    "" + filePath + objName + "####.obj\n\n"\
    "From frame " + str(fromTime) + " to " + str(toTime) + " "
proceedBool = c4d.gui.QuestionDialog(questionDialogText)

if proceedBool == True:

    # Loop through animation and export frames
    for x in range(0,animLength):

        # change frame, redraw view
        moveTime = c4d.BaseTime(fromTime,docFps) + c4d.BaseTime(x)
        doc.SetTime(moveTime)
        c4d.EventAdd(c4d.EVENT_FORCEREDRAW)
        c4d.DrawViews(c4d.DRAWFLAGS_FORCEFULLREDRAW)

        # update status bar
        c4d.StatusSetText("Exporting " + str(x) + " of " + str(animLength))
        c4d.StatusSetBar(100.0*x/animLength)

        # add buffer 0001
        bufferedNumber = str(doc.GetTime().GetFrame(docFps))
        if len(bufferedNumber) < 4:
```

기상청 웹 사이트의 날씨 데이터 가져오기

- ☛ 전국 중기예보 데이터를 RSS를 통해 가져와 주기적으로 데이터베이스에 저장하기

(https://www.weather.go.kr/weather/lifenindustry/sevice_rss.jsp)

중기예보

중기 예보	전국	RSS	서울·경기도	RSS	충청북도	RSS	충청남도	RSS
	강원도	RSS	경상북도	RSS	경상남도	RSS	제주특별자치도	RSS

Result Grid

Filter Rows:

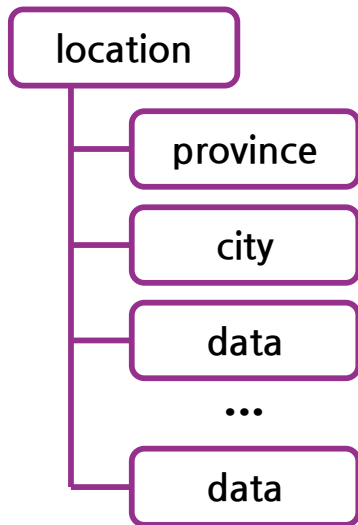
id	city	tmef	wf	tmn	tmx
1	서울	2020-01-08 00:00	흐리고 비	4	6
2	서울	2020-01-08 12:00	구름많음	4	6
3	서울	2020-01-09 00:00	맑음	-3	4
4	서울	2020-01-09 12:00	흐림	-3	4
5	서울	2020-01-10 00:00	흐림	-3	5
6	서울	2020-01-10 12:00	흐림	-3	5
7	서울	2020-01-11 00:00	맑음	-2	5
8	서울	2020-01-11 12:00	흐림	-2	5

1. 데이터 파싱 Step1

날씨 데이터의 패턴을 찾는다.

```

</header>
▼<body>
  ▼<location wl_ver="3">
    <province>서울 · 인천 · 경기도</province>
    <city>서울</city>
    ▼<data>
      <mode>A02</mode>
      <tmEf>2020-01-08 00:00</tmEf>
      <wf>흐리고 비</wf>
      <tmn>4</tmn>
      <tmx>6</tmx>
      <reliability/>
      <rnSt>100</rnSt>
    </data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
    ▶<data>...</data>
  </location>
  ▼<location wl_ver="3">
    <province>서울 · 인천 · 경기도</province>
    <city>인천</city>
    ▶<data>...</data>
    ▶<data>...</data>
  
```



<http://www.weather.go.kr/weather/forecast/mid-term-rss3.jsp?stnId=108>

1. 데이터 파싱 Step2

각 도시의 데이터를 출력한다.

#도시 출력

```
import requests  
from bs4 import BeautifulSoup
```

```
req = requests.get("http://www.weather.go.kr/weather/forecast/mid-term-rss3.jsp?stnId=108")  
html = req.text
```

```
soup = BeautifulSoup(html, 'lxml')  
for i in soup.find_all('location'):  
    print(i.find('city'))
```

XML 파서

```
<city>나주</city>  
<city>전주</city>  
<city>군산</city>  
<city>정읍</city>  
<city>남원</city>  
<city>고창</city>  
<city>무주</city>  
<city>부산</city>
```

1. 데이터 파싱 Step3

날씨 데이터를 출력한다.

```
#날씨 출력
import requests
from bs4 import BeautifulSoup

req = requests.get("http://www.weather.go.kr/weather/forecast/mid-term-rss3.jsp?stnId=108")
html = req.text

soup = BeautifulSoup(html, 'lxml')
seoul = soup.find('location')

#tmef(날짜), wf(날씨), tmn(최저기온), tmx(최고기온)
for i in seoul.find_all('data'):
```

첫 번째 지역(서울)

```
<tmef>2020-01-08 00:00</tmef>
<wf>흐리고 비</wf>
<tmn>4</tmn>
<tmx>6</tmx>
<tmef>2020-01-08 12:00</tmef>
<wf>구름많음</wf>
<tmn>4</tmn>
<tmx>6</tmx>
<tmef>2020-01-09 00:00</tmef>
<wf>맑음</wf>
<tmn>-3</tmn>
```

2. 데이터 가공 Step1

파싱 데이터를 분석하고 자료구조를 설계한다.

`<city>서울</city>`

`<tmef>2020-01-08 00:00</tmef>
<wf>흐리고 비</wf>
<tmn>4</tmn>
<tmx>6</tmx>`

.....

`<tmef>2020-01-14 00:00</tmef>
<wf>맑음</wf>
<tmn>-3</tmn>
<tmx>3</tmx>
<tmef>2020-01-15 00:00</tmef>
<wf>맑음</wf>
<tmn>-4</tmn>
<tmx>3</tmx>`



하나의 도시, 여러 개의 날씨 데이터

1. 사전 자료형 key : 도시 , value : 날씨
2. 리스트 자료형 : 날씨
→ { '도시' : [[날씨1],[날씨2] ... [날씨 n]] }

2. 데이터 가공 Step2

반복문을 활용해 데이터를 입력한다.

#데이터 가공

```
import requests
from bs4 import BeautifulSoup
```

```
req = requests.get("http://www.weather.go.kr/weather/forecast/mid-term-rss3.jsp?stnId=108")
html = req.text
```

```
soup = BeautifulSoup(html, 'lxml')
```

```
weather = {}
```

```
#city, tmef, wf, tmn, tmx
```

```
for i in soup.find_all('location'):
```

```
    weather[i.find('city').text] = []
```

```
    for j in i.find_all('data'):
```

```
        temp = []
```

```
        temp.append(j.find('tmef').text)
```

```
        temp.append(j.find('wf').text)
```

```
        temp.append(j.find('tmn').text)
```

```
        temp.append(j.find('tmx').text)
```

```
    weather[i.find('city').string].append(temp)
```

```
print(weather.keys())
```

```
dict_keys(['서울', '인천', '수원', '파주', '이천', '평택', '춘천', '원주', '강릉', '대전', '세종', '홍성', '청주', '충주', '영동', '광주', '목포', '여수', '순천', '광양', '나주', '전주', '군산', '정읍', '남원', '고창', '무주', '부산', '울산', '창원', '진주', '거창', '통영', '대구', '안동', '포항', '경주', '울진', '울릉도', '제주', '서귀포'])
```

```
print(weather['서울'])
```

```
[['2020-01-08 00:00', '호리고 비', '4', '6'], ['2020-01-08 12:00', '구름맑음', '4', '6'], ['2020-01-09 00:00', '맑음', '-3', '4'], ['2020-01-09 12:00', '흐림', '-3', '4'], ['2020-01-10 00:00', '흐림', '-3', '5'], ['2020-01-10 12:00', '흐림', '-3', '5'], ['2020-01-11 00:00', '맑음', '-2', '5'], ['2020-01-11 12:00', '흐림', '-2', '5'], ['2020-01-12 00:00', '흐림', '-2', '4'], ['2020-01-12 12:00', '맑음', '-2', '4'], ['2020-01-13 00:00', '맑음', '-3', '4'], ['2020-01-14 00:00', '맑음', '-3', '3'], ['2020-01-15 00:00', '맑음', '-4', '3']]
```

LESSON

데이터 베이스 입력

```
animLength = toTime - fromTime

# prompt user for directory
filePath = c4d.storage.SaveDialog()
filePath, objName = os.path.split(filePath)
objName = objName + "-"
filePath = filePath + "\\ "

# check for confirmation
questionDialogText = "Obj Sequence will be saved as:\n\n" \
    "" + filePath + objName + "###.obj\n\n" \
    "From frame " + str(fromTime) + " to " + str(toTime) + " "
proceedBool = c4d.gui.QuestionDialog(questionDialogText)

if proceedBool == True:

    # loop through animation and export frames
    for x in range(0,animLength):

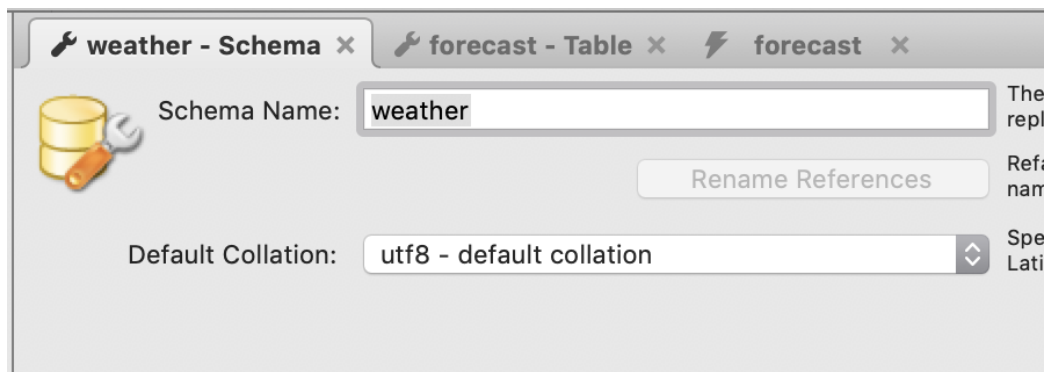
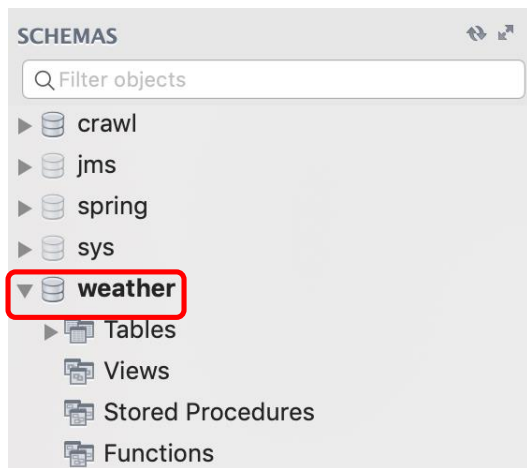
        # change frame, redraw view
        moveTime = c4d.BaseTime(fromTime,docFps) + c4d.BaseTime(x,docFps)
        doc.SetTime(moveTime)
        c4d.EventAdd(c4d.EVENT_FORCEREDRAW)
        c4d.DrawViews(c4d.DRAWFLAGS_FORCEFULLREDRAW)

        # update status bar
        c4d.StatusSetText("Exporting " + str(x) + " of " + str(animLength))
        c4d.StatusSetBar(100.0*x/animLength)

        # get buffer pool
        bufferedNumber = str(doc.GetTime().GetFrame(docFps))
        if len(bufferedNumber) > 10:
```

1. 스키마 및 테이블 생성 Step1

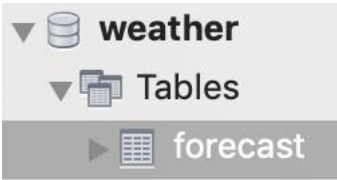
weather 스키마를 생성한다.



forecast 테이블을 생성한다.



- ① id
- ② city
- ③ tmef
- ④ wf
- ⑤ tmn
- ⑥ tmx

[illegible]

2. DB에 연결하기

pymysql 모듈을 이용해 MySQL 데이터 베이스와 연결한다.

```
conn = pymysql.connect(  
    host=dbURL, port=dbPort, user=dbUser, passwd=dbPass, db='weather',  
    charset='utf8', use_unicode=True  
)
```



pymysql 모듈을 통해 MySQL 데이터 베이스에 접근할 수 있음

- ① 설치: pip install pymysql
- ② MySQL 데이터 베이스와 연결 : connect()
- ③ 데이터 베이스에 작업을 전달하기 위한 객체 생성 : cursor()
- ④ 질의문 전달 : execute()
- ⑤ 실행 : commit()
- ⑥ SELECT한 결과를 가져옴 : fetchone(), fetchall()

```
cur = conn.cursor()  
cur.execute("SQL")  
conn.commit()
```

```
cur = conn.cursor()  
cur.execute("SELECT SQL문")  
result = cur.fetchone()  
conn.commit()
```

3. 데이터 입력 INSERT 명령문을 통해 각 필드의 값을 입력시킨다.

```
insert_weather = "INSERT INTO `weather`.`forecast` (`city`, `tmef`, `wf`, `tmn`, `tmx`) VALUES (%s,%s,%s,%s,%s)"
```

매개변수로
값을 전달



INSERT 질의문의 형식

INSERT INTO `스키마`.`테이블` (필드1, 필드2 ...) VALUES (값1, 값2 ...)

3. 데이터 입력

INSERT 명령문을 통해 각 필드의 값을 입력시킨다.

```
#db 입력
# -*- coding: utf-8 -*-
import pymysql

dbURL="127.0.0.1"
dbPort=3306
dbUser="root"
dbPass="비밀번호"

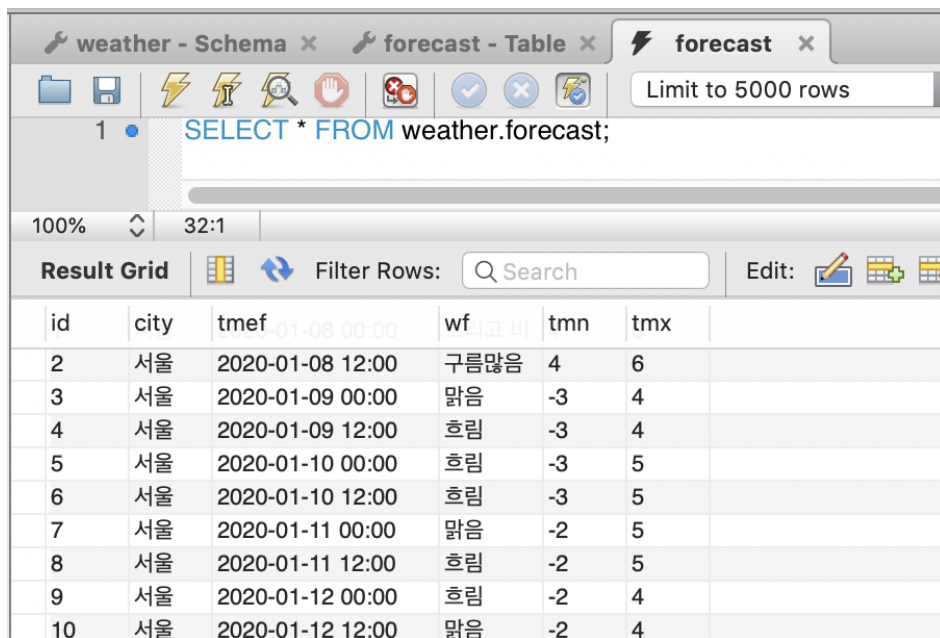
conn = pymysql.connect(
    host=dbURL, port=dbPort, user=dbUser, passwd=dbPass, db='weather',
    charset='utf8', use_unicode=True
)

insert_weather = "INSERT INTO `weather`.`forecast` (`city`, `tmef`, `wf`, `tmn`, `tmx`) VALUES (%s,%s,%s,%s,%s)"

for i in weather:
    for j in weather[i]:
        cur = conn.cursor()
        cur.execute(insert_weather,(i,j[0],j[1],j[2],j[3]))
        conn.commit()
```

4. 데이터 조회

SELECT 명령문을 통해 입력된 데이터를 조회해 확인한다.



The screenshot shows a database application window with three tabs: 'weather - Schema', 'forecast - Table', and 'forecast'. The 'forecast' tab is active, displaying a SQL query editor with the text 'SELECT * FROM weather.forecast;'. Below the editor, a 'Result Grid' shows the query results. The grid has columns for 'id', 'city', 'tmef', 'wf', 'tmn', and 'tmx'. The data is filtered to show 10 rows, with a 'Limit to 5000 rows' option visible in the top right. The results show weather data for Seoul from January 8th to 12th, 2020.

id	city	tmef	wf	tmn	tmx
2	서울	2020-01-08 12:00	구름많음	4	6
3	서울	2020-01-09 00:00	맑음	-3	4
4	서울	2020-01-09 12:00	흐림	-3	4
5	서울	2020-01-10 00:00	흐림	-3	5
6	서울	2020-01-10 12:00	흐림	-3	5
7	서울	2020-01-11 00:00	맑음	-2	5
8	서울	2020-01-11 12:00	흐림	-2	5
9	서울	2020-01-12 00:00	흐림	-2	4
10	서울	2020-01-12 12:00	맑음	-2	4

LESSON

자동 크롤러 만들기

```
animLength = toTime - fromTime

# Ask user for directory
filePath = c4d.storage.SaveDialog()
filePath, objName = os.path.split(filePath)
objName = objName + "-"
filePath = filePath + "\\ "

# Ask for confirmation
questionDialogText = "Obj Sequence will be saved as:\n\n"\
    "" + filePath + objName + "###.obj\n\n"\
    "From frame " + str(fromTime) + " to " + str(toTime) + " "
proceedBool = c4d.gui.QuestionDialog(questionDialogText)

if proceedBool == True:

    # Loop through animation and export frames
    for x in range(0,animLength):

        # change frame, redraw view
        moveTime = c4d.BaseTime(fromTime,docFps) + c4d.BaseTime(x,docFps)
        doc.SetTime(moveTime)
        c4d.EventAdd(c4d.EVENT_FORCEREDRAW)
        c4d.DrawViews(c4d.DRAWFLAGS_FORCEFULLREDRAW)

        # update status bar
        c4d.StatusSetText("Exporting " + str(x) + " of " + str(animLength))
        c4d.StatusSetBar(100.0*x/animLength)

        # get buffer pool
        bufferedNumber = str(doc.GetTime().GetFrame(docFps-x))
        if len(bufferedNumber) < 4:
```

1. 작업 단위 코드 재구성

schedule 모듈 활용 시 특정 시간마다 특정 작업을 수행하도록 크롤링 코드를 작업 단위로 재구성한다.

```
#데이터 가공
import requests
from bs4 import BeautifulSoup

req = requests.get("http://www.weather.go.kr/weather/forecast/mid-term-rss3.jsp?stnId=108")
html = req.text

soup = BeautifulSoup(html, 'lxml')

weather = {}
#city, tmeff, wf, tmn, tmx
for i in soup.find_all('location'):
    weather[i.find('city').text] = []
    for j in i.find_all('data'):
        temp = []
        temp.append(j.find('tmeff').text)
        temp.append(j.find('wf').text)
        temp.append(j.find('tmn').text)
        temp.append(j.find('tmx').text)
        weather[i.find('city').text].append(temp)
```

```
req = requests.get("http://www.weather.go.kr/weather/forecast/mid-term-rss3.jsp?stnId=108")
html = req.text

soup = BeautifulSoup(html, 'lxml')

weather = {}
#city, tmeff, wf, tmn, tmx
for i in soup.find_all('location'):
    weather[i.find('city').text] = []
    for j in i.find_all('data'):
        temp = []
        temp.append(j.find('tmeff').text)
        temp.append(j.find('wf').text)
        temp.append(j.find('tmn').text)
        temp.append(j.find('tmx').text)
        weather[i.find('city').text].append(temp)
```

주기적으로 실행되어야 할
코드를 함수로 구성

```
#db 연결
#-*- coding: utf-8 -*-
import pymysql

dbURL="127.0.0.1"
dbPort=3306
dbUser="root"
dbPass="비밀번호"
```

```
conn = pymysql.connect(
    host=dbURL, port=dbPort, user=dbUser, passwd=dbPass, db='weather',
    charset='utf8', use_unicode=True
)
```

```
insert_weather = "INSERT INTO 'weather'.'forecast' ('city', 'tmeff', 'wf', 'tmn', 'tmx') VALUES (%s, %s, %s, %s, %s)"

for i in weather:
    for j in weather[i]:
        cur = conn.cursor()
        cur.execute(insert_weather, (i, j[0], j[1], j[2], j[3]))
        conn.commit()
```

```
for i in weather:
    for j in weather[i]:
        cur = conn.cursor()
        cur.execute(insert_weather, (i, j[0], j[1], j[2], j[3]))
        conn.commit()
```

1. 작업 단위 코드 재구성

schedule 모듈 활용 시 특정 시간마다 특정 작업을 수행하도록 크롤링 코드를 작업 단위로 재구성한다.

```
# -*- coding: utf-8 -*-
import pymysql
import requests
from bs4 import BeautifulSoup
```

```
dbURL="127.0.0.1"
dbPort=3306
dbUser="root"
dbPass="비밀번호"
```

```
conn = pymysql.connect(
    host=dbURL, port=dbPort,
    charset='utf8', use_unicode=True
)
```

```
insert_weather = "INSERT INTO
```

```
def job():
    req = requests.get("http://www.weather.go.kr/weather/forecast/mid-term-rss3.jsp?stnId=108")
    html = req.text
```

```
    soup = BeautifulSoup(html, 'lxml')
```

```
    weather = {}
```

```
    for i in soup.find_all('location'):
        weather[i.find('city').text] = []
        for j in i.find_all('data'):
            temp = []
            temp.append(j.find('tmef').text)
            temp.append(j.find('wf').text)
            temp.append(j.find('tmn').text)
            temp.append(j.find('tmx').text)
            weather[i.find('city').string].append(temp)
```

```
    for i in weather:
        for j in weather[i]:
            cur = conn.cursor()
            cur.execute(insert_weather, (i,j[0],j[1],j[2],j[3]))
            conn.commit()
```

```
def job():
    req = requests.get("http://www.weather.go.kr/weather/forecast/mid-term-rss3.jsp?stnId=108")
    html = req.text

    soup = BeautifulSoup(html, 'lxml')

    weather = {}

    for i in soup.find_all('location'):
        weather[i.find('city').text] = []
        for j in i.find_all('data'):
            temp = []
            temp.append(j.find('tmef').text)
            temp.append(j.find('wf').text)
            temp.append(j.find('tmn').text)
            temp.append(j.find('tmx').text)
            weather[i.find('city').string].append(temp)

    for i in weather:
        for j in weather[i]:
            cur = conn.cursor()
            cur.execute(insert_weather, (i,j[0],j[1],j[2],j[3]))
            conn.commit()
```

job() 함수만
주기적으로 호출

2. 스케줄 설정

매주 특정 시간에 크롤링 및 데이터가 새로 입력되도록 한다.

```
import schedule

def job():
    ....

schedule.every().monday.at("06:00").do(job)

while True:
    schedule.run_pending()
```



주간 예보(중기예보)이기 때문에
매주 월요일 오전 6:00에
job () 함수가 실행되도록 함